

1. Como estudar esse material

A lógica que encontrei nos arquivos é:

Fundamentos → Ciclo de vida → Teste estático → Técnicas de teste → Gerenciamento → Ferramentas

Essa ordem não é aleatória.

Primeiro você entende **o que é teste e por que ele existe**. Depois entende **quando e onde o teste acontece no desenvolvimento**. Em seguida aprende **como encontrar problemas sem executar o software**. Aí entram as técnicas para **derivar casos de teste**. Depois você aprende a **planejar, priorizar, medir e gerenciar os testes**. Por último aparecem as ferramentas que dão suporte a tudo isso.

O próprio syllabus organiza o conteúdo nesses seis capítulos, começando por Fundamentos, passando por SDLC e Teste Estático, até Análise/Modelagem, Gerenciamento e Ferramentas.

Mapa de páginas

Conteúdo	Páginas do Livro
1. Fundamentos de Teste	14–20
2. Testes ao longo do SDLC	23–28
3. Teste Estático	30–33
4. Análise e Modelagem de Teste	35–41
5. Gerenciamento das Atividades de Teste	42–51
6. Ferramentas de Teste	53–54

O índice do livro confirma essa distribuição.

Um detalhe muito importante: nos dois simulados, **cada uma das 40 questões vale 1 ponto** e a distribuição dos capítulos é exatamente a mesma:

Capítulo	Questões em A	Questões em B
1 – Fundamentos	8	8

2 – SDLC	6	6
3 – Estático	4	4
4 – Técnicas	11	11
5 – Gerenciamento	9	9
6 – Ferramentas	2	2

Além disso, nos dois conjuntos há **8 questões K1, 24 K2 e 8 K3**. O gabarito de B mostra explicitamente a distribuição dos objetivos de aprendizagem e dos níveis K; o gabarito de A apresenta a mesma estrutura.

Isso significa que os capítulos 4 e 5, juntos, ocupam **20 das 40 questões dos dois simulados**. E isso combina com o próprio livro: o Capítulo 4 recebe 390 minutos de conteúdo e o Capítulo 5, 335 minutos.

2. COMO AS QUESTÕES DOS SIMULADOS FUNCIONAM

Antes de entrar no conteúdo, existe uma coisa que você precisa entender sobre o estilo.

As questões não ficam apenas perguntando:

“O que é X?”

Frequentemente elas apresentam **um cenário** e perguntam qual conceito se aplica.

Por exemplo, o Simulado A coloca situações como:

- equipe desenvolvendo aplicativo;
- mudança em software;
- testes de regressão;
- critérios de aceite;
- defeitos encontrados;
- dependências entre testes;
- estimativas de esforço.

O Simulado B faz o mesmo, inclusive trazendo tabelas e pequenos problemas para serem analisados. O documento informa que as 40 questões do modelo estão organizadas segundo os objetivos de aprendizagem, algo que **não necessariamente ocorrerá dessa forma em uma prova real**.

Portanto, estudar somente por definição é perigoso.

Você precisa conseguir fazer:

“Eu sei o conceito” → “Eu reconheço o conceito dentro de um cenário.”

K1, K2 e K3

O Livro explica:

K1 = lembrar

Você precisa reconhecer ou recordar alguma informação.

Exemplo:

Qual é um exemplo de ferramenta de colaboração?

É muito mais memória.

K2 = compreender

Você precisa entender a situação e identificar o conceito correto.

Exemplo:

Uma equipe encontrou determinado problema. Qual princípio de teste explica essa situação?

K3 = aplicar

Você precisa utilizar o conhecimento.

É onde aparecem coisas como:

- calcular;
- montar casos de teste;
- interpretar uma tabela;
- determinar cobertura;
- ordenar casos de teste;
- calcular esforço;
- analisar um cenário.

O livro define exatamente essa divisão: K1 lembrar, K2 compreender e K3 aplicar.

E isso aparece fortemente nos gabaritos. Por exemplo, no Set B, as quatro técnicas de caixa-preta — EP, BVA, tabela de decisão e transição de estado — aparecem como **K3**.

3. CAPÍTULO 1 — FUNDAMENTOS DE TESTE

Livro: páginas 14–20

O capítulo começa estabelecendo uma ideia fundamental:

Teste não é simplesmente executar o programa.

O teste é um conjunto de atividades destinadas a **descobrir defeitos e avaliar a qualidade dos artefatos de software**.

Isso já aparece na página 14, junto com a diferença entre teste dinâmico e estático, verificação e validação.

3.1 O que é teste?

Teste dinâmico

O software é **executado**.

Você fornece entradas, observa o comportamento e compara:

Resultado esperado × Resultado real

Exemplo:

Sistema deve permitir login.

Entrada:

Usuário: joao

Senha: 123456

Esperado:

Acesso concedido

Real:

Acesso negado

Existe uma anomalia que precisa ser investigada.

3.2 Teste estático

O software **não precisa ser executado**.

Pode-se analisar:

- requisitos;
- histórias de usuário;
- código;
- arquitetura;
- documentação;
- casos de teste;
- contratos;
- modelos.

É um assunto que depois será aprofundado no Capítulo 3.

3.3 Verificação × validação

Esse é um conceito que você precisa separar muito bem.

Verificação

Pergunta:

“Construímos o produto de acordo com o que foi especificado?”

Foco nos requisitos especificados.

Validação

Pergunta:

“Construímos aquilo que realmente atende às necessidades do usuário?”

Foco na necessidade e expectativa do stakeholder.

Exemplo

Requisito:

O sistema deve permitir cadastrar clientes.

O sistema permite cadastrar clientes.

Verificação: está funcionando de acordo com o requisito.

Mas imagine que o processo real da empresa exige também que o cliente possa anexar documentos.

Se ninguém considerou isso:

Validação: o sistema pode estar correto em relação ao requisito e ainda não atender à necessidade real.

Essa distinção é muito importante porque se relaciona diretamente ao princípio da **falácia da ausência de defeitos**, que veremos adiante.

3.4 Objetivos do teste

Os objetivos típicos incluem:

- avaliar produtos de trabalho;
- detectar defeitos e falhas;
- verificar requisitos;
- avaliar cobertura;
- reduzir riscos;
- fornecer informações aos stakeholders;
- aumentar a confiança na qualidade;
- validar se o produto atende às necessidades dos stakeholders.

O ponto importante é:

Teste não existe somente para encontrar bugs.

Ele também produz **informação para tomada de decisão**.

Essa ideia aparece diretamente nas questões de Fundamentos dos dois simulados. No Set A, por exemplo, a questão 1 trabalha o objetivo de reduzir risco e aumentar confiança na qualidade; no gabarito ela é identificada como **FL-1.1.1**.

3.5 Teste × depuração

Essa é uma pegadinha clássica.

Teste

Tem como objetivo **encontrar defeitos/falhas e avaliar o software**.

Depuração

Tem como objetivo **encontrar a causa do problema e corrigi-la**.

Pense na sequência:

Teste



Falha observada



Depuração



Encontrar causa/defeito



Corrigir



Teste de confirmação



Teste de regressão

A depuração, portanto, **não é considerada uma atividade de teste**.

O livro descreve o processo de depuração como:

1. reproduzir a falha;
2. diagnosticar;
3. corrigir a causa.

3.6 QA × QC × Teste

Essa diferença aparece diretamente no Simulado B.

QA — Garantia da Qualidade

Foco:

processo

É uma abordagem preventiva.

Exemplo:

Melhorar o processo de desenvolvimento para evitar que determinados erros aconteçam.

QC — Controle da Qualidade

Foco:

produto

É uma abordagem corretiva.

Teste

O teste é uma forma de **Controle de Qualidade**.

Portanto:

QA

└─ foco em processos

QC

└─ foco no produto

└─ Testes

O gabarito B reforça exatamente essa relação: QA trabalha com melhoria/implementação de processos, enquanto os testes fazem parte do controle da qualidade.

Memorize assim:

QA tenta melhorar o processo para prevenir problemas.

QC verifica o produto para detectar problemas.

Teste é parte do QC.

3.7 Erro × Defeito × Falha × Causa-raiz

Esse é um dos conceitos mais importantes do Capítulo 1.

A sequência básica é:

Pessoa comete um ERRO



surge um DEFEITO



defeito é executado



pode acontecer uma FALHA

Erro

Engano humano.

Exemplo:

O programador interpreta uma regra incorretamente.

Defeito

Problema introduzido no produto de trabalho.

Exemplo:

O código implementa:

idade > 18

quando deveria ser:

idade >= 18

Falha

Comportamento observado quando o sistema não funciona como esperado.

Exemplo:

Usuário com 18 anos tenta entrar e recebe acesso negado.

Causa-raiz

A causa fundamental que levou ao problema.

Talvez o requisito estivesse ambíguo.

Então poderíamos ter:

Requisito ambíguo



Erro humano



Defeito no código



Falha na execução

O livro também ressalta que falhas não necessariamente são causadas apenas por erros humanos ou defeitos: condições ambientais também podem provocar falhas. A explicação está na página 16.

3.8 Os 7 princípios de teste

Livro: página 16

Você deve saber os sete e, principalmente, **reconhecer situações que exemplificam cada um**.

1. O teste mostra a presença, não a ausência de defeitos

Encontrar defeitos prova que existem defeitos.

Não encontrar defeitos **não prova que não existem defeitos**.

2. Testes exaustivos são impossíveis

Não conseguimos testar:

todas as entradas + todas as combinações + todas as condições.

Por isso usamos técnicas como:

- particionamento de equivalência;
- BVA;
- priorização;
- testes baseados em risco.

O próprio gabarito B usa EP e BVA como exemplo prático desse princípio.

3. Testes antecipados economizam tempo e dinheiro

Quanto mais cedo você encontra o problema, menor tende a ser o custo de corrigi-lo.

Por isso:

Requisito → revisão

Projeto → revisão

Código → análise estática

Código → testes

em vez de esperar:

Software pronto

↓

descobrir problema

4. Os defeitos se agrupam

Uma pequena quantidade de componentes pode concentrar grande parte dos defeitos.

É relacionado ao chamado princípio de Pareto.

5. Os testes se degradam

Se você sempre executar exatamente os mesmos testes, eles podem deixar de encontrar novos defeitos.

Por isso devemos:

- modificar testes;
- criar novos testes;
- alterar dados;
- variar condições.

Cuidado: isso não significa que testes repetitivos sejam inúteis. Testes de regressão automatizados são justamente uma situação em que repetir testes é extremamente útil.

6. Os testes dependem do contexto

Não existe uma única maneira correta de testar tudo.

Um:

aplicativo bancário

não necessariamente será testado da mesma maneira que:

jogo mobile

ou:

sistema hospitalar

7. Falácia da ausência de defeitos

Mesmo corrigindo todos os defeitos conhecidos, o produto pode continuar sendo inadequado.

Porque pode:

- não atender à necessidade do usuário;

- não atingir o objetivo de negócio;
- ser inferior a uma solução concorrente.

Por isso **verificação não substitui validação**.

O gabarito do Set A usa diretamente a situação de repetir os mesmos testes e não encontrar novos defeitos como exemplo do princípio 5.

3.9 Atividades de teste

Livro: páginas 17–18

Você precisa diferenciar:

Atividade	Pergunta mental
Planejamento	O que queremos alcançar e como?
Monitoramento e controle	Estamos seguindo o plano?
Análise	O que testar?
Modelagem	Como testar?
Implementação	Preparar casos, dados, ambiente etc.
Execução	Executar os testes
Conclusão	Encerrar, analisar resultados e aprender

Uma das questões do Set A, por exemplo, pergunta qual atividade trabalha com a identificação de condições de teste. A resposta está ligada à **análise de teste**, não à execução ou ao planejamento.

Regra para decorar

Análise = o que testar.

Modelagem = como testar.

Essa diferenciação é extremamente útil.

3.10 Testware

Testware é o conjunto de produtos de trabalho gerados e utilizados durante o processo de teste.

Exemplos:

- plano de teste;
- casos de teste;
- dados de teste;
- scripts;
- procedimentos;
- registros;
- relatórios;
- critérios de teste;
- ambiente de teste.

A página 18 organiza esses produtos de acordo com as atividades de teste.

3.11 Rastreabilidade

Livro: página 19

Pense:

Requisito
↓
Condição de teste
↓
Caso de teste
↓
Resultado
↓
Defeito

A rastreabilidade permite saber:

“Este requisito foi realmente testado?”

Também ajuda a:

- avaliar cobertura;
- analisar impacto de mudanças;
- realizar auditorias;
- compreender relatórios;

- acompanhar riscos.
-

3.12 Papéis

O syllabus separa dois grandes papéis:

Gerenciamento de teste

Foco maior:

- planejamento;
- monitoramento;
- controle;
- conclusão.

Testador

Foco maior:

- análise;
- modelagem;
- implementação;
- execução.

Essa distinção é explorada diretamente no Simulado A, inclusive com uma questão pedindo quais atividades pertencem principalmente ao papel de teste.

3.13 Habilidades do testador

Livro: página 20

O testador precisa de:

- conhecimento de testes;
- atenção aos detalhes;
- curiosidade;
- pensamento crítico;
- pensamento analítico;
- criatividade;
- comunicação;
- capacidade de trabalhar em equipe;
- conhecimento técnico;

- conhecimento do domínio.

O Simulado A usa justamente uma questão que combina **conhecimento do domínio + trabalho em equipe + pensamento crítico**.

4. CAPÍTULO 2 — TESTES AO LONGO DO CICLO DE VIDA

Livro: páginas 23–28

Aqui começa uma mudança importante:

No Capítulo 1 você aprendeu **o que é teste**.

Agora:

Onde o teste entra no desenvolvimento?

O capítulo trata de SDLC, DevOps, shift-left, TDD/ATDD/BDD, níveis de teste, tipos de teste, confirmação, regressão e manutenção.

4.1 SDLC

SDLC = **Software Development Life Cycle**.

O livro apresenta modelos:

Sequenciais

Exemplos:

- Waterfall;
- V-Model.

Iterativos

Exemplos:

- Spiral;
- Prototipagem.

Incrementais

Entrega-se o software em partes.

O ponto principal:

O SDLC escolhido influencia os testes.

Ele afeta:

- quando testar;
 - quantidade de automação;
 - documentação;
 - técnicas;
 - responsabilidades;
 - níveis de teste.
-

4.2 Uma atividade de desenvolvimento → atividade de teste correspondente

Essa é uma boa prática importante.

Não devemos pensar:

Desenvolvimento



Software pronto



Teste

Mas sim:

Requisitos → revisão/teste

Projeto → revisão/teste

Código → teste

Integração → teste

Sistema → teste

Essa ideia aparece diretamente no Simulado A na questão sobre modelos de SDLC.

4.3 TDD × ATDD × BDD

Esses três costumam confundir.

TDD

Test-Driven Development

O teste orienta a implementação do código.

Fluxo simplificado:

Escreve teste



Escreve código



Faz passar



Refatora

ATDD

Acceptance Test-Driven Development

Parte dos **critérios de aceite**.

A equipe cria testes para orientar o desenvolvimento da história.

BDD

Behavior-Driven Development

Foco:

comportamento desejado.

É comum utilizar:

Dado

Quando

Então

O livro diferencia claramente as três abordagens na página 24.

Macete conceitual

TDD → código

ATDD → aceitação
BDD → comportamento

4.4 DevOps

DevOps aproxima:

Desenvolvimento + Testes + Operações

O objetivo é criar um fluxo integrado.

Entram conceitos como:

- CI — Integração Contínua;
- CD — Entrega Contínua;
- pipeline;
- automação;
- feedback rápido.

Consequência para os testes

DevOps favorece:

- automação;
- testes frequentes;
- feedback rápido;
- regressão automatizada;
- ambientes mais consistentes;
- integração entre desenvolvimento e testes.

O livro trata DevOps diretamente na página 24.

4.5 Shift-left

Pense numa linha:

INÍCIO ----- FIM

Shift-left significa:

trazer determinadas atividades de teste para mais cedo no processo.

Exemplos:

Em vez de

esperar o código ficar pronto.

Fazer

revisar requisitos antes;
criar testes antes;
usar análise estática cedo;
executar testes automatizados no pipeline.

Mas existe uma pegadinha:

Shift-left **não significa abandonar os testes posteriores.**

Ele significa antecipar testes quando possível.

O livro apresenta exatamente essa definição e os exemplos na página 25.

4.6 Retrospectivas

Depois de uma iteração/projeto, a equipe analisa:

O que funcionou?
O que não funcionou?
O que podemos melhorar?

O objetivo é transformar problemas encontrados em **melhoria do processo**.

Por isso uma retrospectiva não é simplesmente:

“Vamos reclamar do que aconteceu.”

É:

Problema identificado



Análise



Ação de melhoria



Aplicação



Nova avaliação

O Simulado A apresenta exatamente uma questão em que a alternativa correta relaciona a retrospectiva a uma lista de tarefas para melhoria contínua.

4.7 Níveis de teste

Livro: página 26

Esse assunto é extremamente importante.

1. Teste de componente

Testa um componente isoladamente.

Geralmente próximo do código.

2. Teste de integração de componentes

Testa:

interfaces e interações entre componentes.

3. Teste de sistema

Avalia o comportamento do sistema como um todo.

Pode envolver:

- funcional;
 - não funcional;
 - ponta a ponta.
-

4. Teste de integração de sistema

Foco:

comunicação do sistema com outros sistemas/serviços externos.

5. Teste de aceite

Foco:

verificar se o sistema atende às necessidades de negócio/usuário.

Pode envolver:

- UAT;
- aceite operacional;
- contratual/regulatório;
- alfa;
- beta.

O Simulado A traz uma questão que pede justamente para relacionar tipos de falha aos níveis de teste.

Forma de pensar

Componente



Componentes juntos



Sistema



Sistema + sistemas externos



Usuário/negócio

4.8 Tipos de teste

Existem quatro categorias apresentadas no syllabus.

Funcional

Pergunta:

O que o sistema faz?

Exemplo:

O botão “Comprar” realiza uma compra?

Não funcional

Pergunta:

Quão bem ele funciona?

Exemplos:

- performance;
 - compatibilidade;
 - usabilidade;
 - confiabilidade;
 - segurança;
 - manutenibilidade;
 - portabilidade.
-

Caixa-preta

Não depende da estrutura interna.

Foco:

comportamento/especificação.

Caixa-branca

Olha a estrutura interna.

Foco:

código, fluxos, estrutura, ramificações etc.

As páginas 26–27 deixam essas diferenças explícitas.

4.9 Confirmação × regressão

Essa é uma das maiores pegadinhas dos simulados.

Teste de confirmação

Pergunta:

“O defeito corrigido realmente foi corrigido?”

Exemplo:

Bug:

Login com senha correta falhava.

Corrigiu.

Teste novamente o problema.

Isso é **confirmação**.

Teste de regressão

Pergunta:

“A alteração criou algum problema em outro lugar?”

Exemplo:

Corrigi login.

Agora preciso verificar:

- cadastro;
- recuperação de senha;
- sessão;
- pagamentos;
- outras funcionalidades afetadas.

Isso é regressão.

O livro explica esses conceitos nas páginas 27–28 e destaca que a análise de impacto ajuda a definir o escopo da regressão.

Decore:

Confirmação = a correção funcionou?

Regressão = a correção quebrou alguma outra coisa?

4.10 Teste de manutenção

Acontece quando um sistema já existente sofre alterações.

Pode ocorrer por:

- correções;
- novos recursos;
- mudança de plataforma;
- migração;
- mudanças de ambiente;
- aposentadoria do sistema.

A extensão do teste depende principalmente de:

risco da mudança

+

tamanho do sistema

+

tamanho da mudança

O conteúdo está na página 28.

5. CAPÍTULO 3 — TESTE ESTÁTICO

Livro: páginas 30–33

Agora entra uma distinção fundamental:

Teste dinâmico

= executa

Teste estático

= não precisa executar

5.1 O que pode ser analisado estaticamente?

Quase qualquer produto de trabalho que possa ser lido/compreendido:

- requisitos;

- código;
- plano de teste;
- casos de teste;
- backlog;
- contratos;
- modelos;
- documentação.

Inclusive requisitos podem ser analisados **antes de existir código**.

5.2 Análise estática

Pode usar ferramentas.

Exemplos:

- análise de código;
 - verificadores;
 - ferramentas de legibilidade;
 - verificadores de padrões.
-

5.3 Revisão

Pode ser manual.

Vários stakeholders analisam um produto de trabalho.

5.4 Por que teste estático é importante?

Ele consegue encontrar cedo problemas como:

- ambiguidade;
- inconsistência;
- contradição;
- requisitos incompletos;
- código inacessível;
- variáveis não declaradas;
- violações de padrões;

- problemas de interface.

Isso tende a reduzir esforço posterior.

5.5 Estático × dinâmico

Estático	Dinâmico
Não executa o software	Executa
Pode encontrar defeito diretamente	Pode provocar falha
Pode analisar documento	Precisa de produto executável para teste dinâmico
Muito útil cedo	Útil durante execução
Revisões/análise estática	Casos de teste executados

A página 31 é particularmente importante para essa comparação.

5.6 Feedback antecipado

Quanto mais tarde o stakeholder percebe:

“Não era isso que eu queria.”

maior pode ser o retrabalho.

Por isso:

Feedback cedo



problema pequeno



correção mais cedo



menos retrabalho

O Simulado A utiliza diretamente essa ideia em questão sobre o benefício do feedback precoce e frequente.

5.7 Processo de revisão

Livro: página 32

As etapas principais são:

1. Planejamento
2. Início da revisão
3. Revisão individual
4. Comunicação/análise dos problemas
5. Correção e relatório

Isso é uma sequência que você precisa reconhecer.

5.8 Papéis da revisão

Gerente

Fornece recursos e decide o que será revisado.

Autor

Produz/corrigi o produto.

Moderador

Facilita a revisão.

Relator

Registra informações e anomalias.

Revisor

Analisa o material.

Líder da revisão

Coordena a revisão como um todo.

5.9 Tipos de revisão

Livro: páginas 32–33

Revisão informal

Pouca formalidade.

Walkthrough

O autor conduz a apresentação passo a passo.

Revisão técnica

Participantes tecnicamente qualificados + moderador.

Inspeção

Mais formal.

Objetivo central:

encontrar o maior número possível de anomalias.

O próprio syllabus destaca que a escolha do tipo de revisão depende do contexto, complexidade, risco, necessidade de auditoria etc.

6. CAPÍTULO 4 — ANÁLISE E MODELAGEM DE TESTE

Livro: páginas 35–41

Esse é o capítulo mais pesado dos simulados.

O gabarito mostra que **as questões 20–23 dos dois conjuntos trabalham diretamente com técnicas caixa-preta em nível K3.**

Aqui você precisa aprender principalmente a **usar a técnica**, e não apenas saber sua definição.

6.1 As três famílias

Caixa-preta

Baseada em:

especificações/comportamento.

Não importa como o código foi implementado.

Caixa-branca

Baseada em:

estrutura interna.

Baseada em experiência

Baseada em:

conhecimento e experiência do testador.

O livro faz essa classificação na página 35.

6.2 Particionamento de Equivalência — EP

Livro: páginas 35–36

A ideia:

separar entradas em grupos que devem ser tratados de maneira equivalente.

Imagine:

idade permitida: 18 a 60 anos.

Podemos criar:

P1: idade < 18

P2: $18 \leq \text{idade} \leq 60$

P3: idade > 60

Temos:

- duas partições inválidas;
- uma partição válida.

Em vez de testar:

0
1
2
3
4
...
100

podemos escolher representantes:

17
30
61

Um representante de cada partição.

Cobertura

100% de cobertura por EP significa testar **todas as partições identificadas**.

6.3 Análise de Valor de Limite — BVA

Livro: página 36

Aqui está um dos assuntos que você precisa saber fazer.

O BVA olha para:

fronteiras das partições.

Para:

18 até 60

os limites são:

18
60

BVA de 2 valores

Considera o limite e o vizinho da partição adjacente.

BVA de 3 valores

Considera:

vizinho

limite

vizinho

Por exemplo, perto do 18:

17 | 18 | 19

Perto do 60:

59 | 60 | 61

O syllabus considera o BVA de 3 valores mais rigoroso que o de 2 valores.

Diferença essencial

EP pergunta:

Quais são os grupos?

BVA pergunta:

Quais são as fronteiras desses grupos?

6.4 Tabela de decisão

Livro: páginas 36–37

Serve principalmente para:

combinações de condições → ações/resultados.

Exemplo:

Condições:

Cliente cadastrado?

Pagamento aprovado?

Podemos ter:

Cadastrado	Pagamento	Ação
V	V	Liberar pedido
V	F	Recusar
F	V	Recusar
F	F	Recusar

Cada combinação corresponde a uma regra.

Quando pensar em tabela de decisão?

Quando a questão trouxer:

“dependendo da combinação de condições, diferentes resultados acontecem.”

Isso é praticamente a definição da técnica.

6.5 Teste de transição de estado

Livro: página 37

Usamos quando o comportamento depende do:

estado atual do sistema + evento.

Exemplo:

Novo

↓ pagar

Pago

↓ enviar

Enviado

↓ entregar

Entregue

Podemos perguntar:

O que acontece se ocorrer o evento “enviar” enquanto o pedido ainda está “Novo”?

Talvez seja uma transição inválida.

Coberturas

O syllabus apresenta, entre outras:

- cobertura de todos os estados;
- cobertura de transições válidas;
- cobertura de todas as transições.

No Simulado B, a questão 23 exige exatamente esse tipo de raciocínio, e o gabarito calcula a cobertura das transições.

6.6 Teste de instrução

Livro: página 38

É caixa-branca.

Você quer verificar quantas **instruções executáveis** foram executadas.

Fórmula:

$$\text{Cobertura} = \frac{\text{instruções executadas}}{\text{total de instruções executáveis}} \times 100$$

Exemplo:

10 instruções existentes.

8 executadas.

$8 \div 10 \times 100 = 80\%$

6.7 Teste de ramificação

Também é caixa-branca.

Agora você não olha somente para instruções.

Olha para:

ramificações possíveis.

Exemplo:

```
if (idade >= 18) {  
    liberar();  
} else {  
    bloquear();  
}
```

Existem duas ramificações:

Verdadeiro

Falso

Para atingir 100% da cobertura de ramificação, precisamos exercitar ambas.

Uma questão muito importante dos simulados é justamente não confundir:

cobertura de instrução

com:

cobertura de ramificação.

O livro afirma que 100% de cobertura de ramificação implica 100% de cobertura de instrução, mas o inverso não é necessariamente verdadeiro.

6.8 Técnicas baseadas em experiência

Suposição de erro

O testador pensa:

“Onde é provável que alguém tenha cometido um erro?”

Pode usar experiência com:

- erros comuns;
 - versões anteriores;
 - sistemas semelhantes;
 - histórico de defeitos.
-

Teste exploratório

Testador:

aprende

+
modela
+
executa
+
avalia

quase simultaneamente.

É especialmente útil quando:

- documentação é insuficiente;
- existe pouco tempo;
- queremos complementar testes mais formais.

O Simulado B cobra diretamente esse conceito.

Checklist

Existe uma lista de pontos a verificar.

Exemplo:

- ☐ Mensagens de erro são compreensíveis?
- ☐ Campos obrigatórios são identificados?
- ☐ Botões possuem comportamento correto?

O item do checklist deve ser **verificável individualmente**.

6.9 Histórias de usuário

Livro: página 40

O syllabus apresenta os **3C**:

Card

Onde a história é registrada.

Conversation

Discussão sobre o que deve ser feito.

Confirmation

Critérios de aceite.

Formato comum:

Como [usuário], quero [objetivo], para [valor].

Exemplo:

Como cliente, quero visualizar meus pedidos anteriores, para acompanhar minhas compras.

6.10 INVEST

Uma boa história deve ser:

I — Independent

Independente.

N — Negotiable

Negociável.

V — Valuable

Valiosa.

E — Estimable

Estimável.

S — Small

Pequena.

T — Testable

Testável.

O objetivo não é decorar apenas a sigla, mas entender o que cada característica significa.

6.11 Critérios de aceite

Livro: página 41

Critério de aceite é:

uma condição que a implementação precisa atender para ser aceita.

Dois formatos aparecem bastante:

Orientado a cenários

Dado...

Quando...

Então...

Orientado por regras

Regras/checklists/tabelas de entrada e saída.

O Simulado B usa exatamente um exemplo no formato:

Considerando...

Quando...

Em seguida...

e pede que você reconheça o tipo de critério.

6.12 ATDD

Esse é importante porque aparece como **K3 nos dois simulados**.

ATDD:

testes são criados a partir dos critérios de aceite antes da implementação da história.

Fluxo:

História



Discussão/Workshop



Critérios de aceite



Casos de teste



Implementação



Execução

A questão 29 de ambos os conjuntos avalia esse conceito como K3.

7. CAPÍTULO 5 — GERENCIAMENTO DAS ATIVIDADES DE TESTE

Livro: páginas 42–51

Esse é o segundo grande bloco dos simulados.

7.1 Plano de teste

Livro: páginas 42–43

Um plano de teste define:

- objetivos;
- recursos;
- processos;
- cronograma;
- escopo;
- riscos;
- abordagem;
- responsabilidades;
- comunicação;
- ambiente;
- métricas;
- orçamento.

A ideia:

“Como vamos realizar os testes?”

7.2 Planejamento de liberação × iteração

Release planning

Olha para a **liberação do produto**.

Envolve:

- backlog;
- histórias;

- riscos;
- esforço;
- abordagem de teste.

Iteration planning

Olha para uma **iteração específica**.

Envolve:

- histórias da iteração;
 - tarefas;
 - testabilidade;
 - esforço;
 - riscos detalhados;
 - aspectos funcionais/não funcionais.
-

7.3 Critérios de entrada × saída

Esse é um dos temas mais recorrentes.

Critério de entrada

Pergunta:

“O que precisa estar pronto para começar?”

Exemplo:

Ambiente disponível
Dados disponíveis
Casos prontos
Recursos disponíveis

Critério de saída

Pergunta:

“O que precisa ser alcançado para terminar?”

Exemplo:

Cobertura mínima alcançada
Testes planejados executados

Defeitos relatados

No Ágil

Entrada:

DoR — Definition of Ready

Saída:

DoD — Definition of Done

O syllabus explica essa relação na página 44.

7.4 Estimativa de esforço

Livro: página 44

Aqui aparecem questões K3.

Existem quatro técnicas principais no material.

1. Estimativa baseada em índices/proporções

Usa dados históricos.

Exemplo:

Projetos anteriores indicam:

Teste = 10% do desenvolvimento

Novo projeto:

Desenvolvimento = R\$ 800.000

Então:

$800.000 \times 0,10 = 80.000$

A questão 31 do Simulado B trabalha justamente com estimativa baseada em proporções. O gabarito a classifica como **FL-5.1.4 K3**.

2. Extrapolação

Usa dados obtidos no próprio projeto.

Exemplo:

Últimas três iterações:

10h

12h

11h

Estimativa da próxima usando a média:

$$10+12+11=33 \Rightarrow \frac{33}{3}=11h$$

3. Wideband Delphi

Especialistas estimam individualmente.

Depois:

estimativas

↓

comparação

↓

discussão

↓

nova estimativa

↓

consenso

O Planning Poker é uma variante dessa abordagem.

4. Estimativa de três pontos

Tem:

- otimista = **o**
- mais provável = **m**
- pessimista = **p**

Fórmula:

$$E = \frac{o + 4m + p}{6}$$

Exemplo do próprio livro:

$$o = 6$$

$m = 9$
 $p = 18$

Então:

$$E = 6 + 4(9) + 18 \quad E = \frac{6 + 4(9) + 18}{6} \quad E = 606 = 10 \quad E = \frac{60}{6} = 10$$

Desvio:

$$SD = p - o \quad SD = \frac{p - o}{6} \quad SD = 18 - 66 = 2 \quad SD = \frac{18 - 6}{6} = 2$$

Resultado:

10 ± 2 horas

7.5 Priorização dos casos de teste

Livro: página 45

Tem três estratégias importantes.

Baseada em risco

Testa primeiro os riscos mais importantes.

Baseada em cobertura

Testa primeiro aquilo que proporciona maior cobertura.

Baseada em requisitos

Testa primeiro os requisitos mais importantes.

Mas existe uma pegadinha:

Dependência entre testes.

Imagine:

TC1 = buscar produto
TC2 = visualizar produto
TC3 = adicionar ao carrinho
TC4 = comprar

Mesmo que TC4 tenha prioridade altíssima, ele depende dos anteriores.

Então:

TC1
↓
TC2
↓
TC3
↓
TC4

A dependência pode obrigar você a executar um teste de prioridade menor antes de outro de prioridade maior.

A questão 32 do Simulado B trabalha exatamente essa combinação:

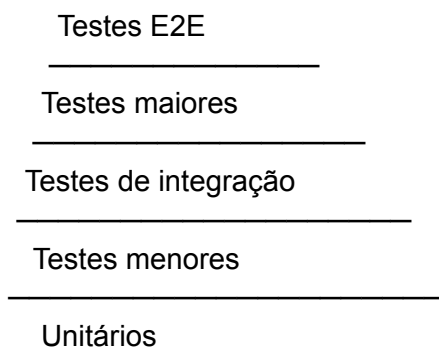
prioridade + dependência lógica.

O gabarito identifica a questão como **FL-5.1.5 K3**.

7.6 Pirâmide de teste

Livro: página 45

A ideia é:



Quanto mais baixo:

- menores;
- mais rápidos;
- mais isolados;
- geralmente mais numerosos.

Quanto mais alto:

- mais complexos;
- mais lentos;
- maior escopo;
- normalmente menos numerosos.

7.7 Quadrantes de teste

Livro: páginas 45–46

São quatro quadrantes.

Quadrante	Característica
Q1	Tecnologia + apoia equipe
Q2	Negócio + apoia equipe
Q3	Negócio + avalia produto
Q4	Tecnologia + avalia produto

Q1

- testes de componente;
- integração de componentes.

Q2

- testes funcionais;
- histórias de usuário;
- API;
- exemplos;
- simulações.

Q3

- exploratório;
- usabilidade;
- aceite do usuário.

Q4

- smoke;
- testes não funcionais, exceto usabilidade.

A questão 33 do Simulado B pede diretamente a identificação do Q1. O gabarito classifica como **FL-5.1.7 K2**.

Um jeito fácil de pensar

Q1 = dev/teste técnico

Q2 = ajudar a construir pelo olhar do negócio

Q3 = usuário avaliando

Q4 = tecnologia avaliando produto

7.8 Gerenciamento de riscos

Livro: páginas 46–48

Primeiro:

Risco = possibilidade de algo causar um efeito adverso.

Possui:

Probabilidade

Chance de ocorrer.

Impacto

Consequência caso ocorra.

No modelo quantitativo:

$\text{Risco} = \text{Probabilidade} \times \text{Impacto}$

Exemplo:

Probabilidade = 0,4

Impacto = 10

$0,4 \times 10 = 4$

Quanto maior o risco, maior tende a ser a necessidade de tratamento.

7.9 Risco de projeto × risco de produto

Risco de projeto

Relacionado ao **projeto**.

Exemplos:

- falta de pessoal;

- atraso;
- orçamento;
- fornecedor;
- ferramenta;
- comunicação.

Risco de produto

Relacionado ao **produto**.

Exemplos:

- funcionalidade errada;
- vulnerabilidade;
- performance ruim;
- cálculo incorreto;
- problema de usabilidade.

Decore assim:

Projeto = conseguimos entregar?

Produto = o que estamos entregando funciona adequadamente?

7.10 Análise e controle de risco

Análise

Identificar

↓

Avaliar

↓

Priorizar

Controle

Mitigar

↓

Monitorar

Para mitigar riscos podemos:

- escolher testadores adequados;
- utilizar independência adequada;
- revisar;
- realizar análise estática;
- usar técnicas de teste;

- aumentar cobertura;
- utilizar tipos apropriados de teste;
- realizar regressão.

O livro descreve essas etapas nas páginas 47–48.

7.11 Métricas

Livro: página 48

É muito importante não confundir os tipos.

Métricas de progresso do projeto

Ex.:

- tarefas concluídas;
- esforço;
- recursos.

Métricas de progresso dos testes

Ex.:

- testes executados;
- aprovados;
- falhos;
- tempo de execução.

Métricas de qualidade do produto

Ex.:

- disponibilidade;
- tempo de resposta;
- MTBF — tempo médio até falha.

Métricas de defeitos

Ex.:

- quantidade;
- prioridade;
- densidade;
- percentual de detecção.

Métricas de cobertura

Ex.:

- cobertura de requisitos;
- cobertura de código.

Métricas de custo

Ex.:

- custo de testes.

A questão 35 do Simulado B é construída justamente para diferenciar **métrica de qualidade do produto** de outras métricas. O gabarito enquadra o tema em FL-5.3.1.

7.12 Relatórios

Livro: páginas 49–50

Relatório de progresso

Serve para informar:

- onde estamos;
- atrasos;
- impedimentos;
- métricas;
- riscos;
- próximos testes.

Relatório de conclusão

Resume:

- o que foi testado;
 - qualidade;
 - desvios;
 - riscos não mitigados;
 - defeitos não corrigidos;
 - lições aprendidas.
-

7.13 Comunicação do status

Pode usar:

- comunicação verbal;
- dashboards;
- CI/CD;
- e-mail;
- chat;
- documentação;
- relatórios formais.

O importante é:

o método deve considerar o público e o contexto.

Por exemplo, uma equipe geograficamente distribuída pode depender mais de comunicação eletrônica e documentação.

A questão 36 do Simulado B explora justamente esse raciocínio.

7.14 Gerenciamento de configuração

Livro: página 50

CM = Configuration Management.

Seu objetivo é:

identificar, controlar, versionar e rastrear itens de configuração.

Exemplos:

- versões do software;
- planos;
- casos de teste;
- scripts;
- ambiente;
- resultados;
- documentação.

Uma das grandes vantagens:

conseguir reproduzir uma determinada configuração.

Exemplo:

Versão sistema 2.1

+

Banco 5.0

+

Biblioteca X 3.2

+

Ambiente Y

Se um resultado antigo precisa ser reproduzido, o gerenciamento de configuração ajuda a recuperar aquela baseline.

A questão 37 do Simulado B avalia exatamente esse conceito.

7.15 Relatório de defeito

Livro: páginas 50–51

Esse é outro tema K3.

Um bom relatório deve permitir:

reproduzir e resolver o problema.

Pode conter:

- identificador;
 - título;
 - data;
 - autor;
 - ambiente;
 - objeto de teste;
 - contexto;
 - passos para reproduzir;
 - resultado esperado;
 - resultado real;
 - severidade;
 - prioridade;
 - status;
 - referência ao teste.
-

Severidade × prioridade

Outra distinção importante.

Severidade

Quanto o problema impacta?

Prioridade

Quão urgente é corrigir?

Não são a mesma coisa.

Um problema pode ter grande impacto, mas baixa prioridade por decisão de negócio.

Ou ter impacto relativamente pequeno, mas alta prioridade por afetar uma funcionalidade muito usada.

A questão 38 do Simulado B apresenta um log de execução e exige reconhecer **qual descrição seria adequada para um relatório de defeitos**. O gabarito associa a questão a FL-5.5.1 K3.

8. CAPÍTULO 6 — FERRAMENTAS DE TESTE

Livro: páginas 53–54

É um capítulo curto, mas aparece nos dois simulados.

8.1 Tipos de ferramentas

O syllabus apresenta, entre outras:

Ferramentas de gerenciamento

Gerenciam:

- requisitos;
- testes;
- defeitos;
- configuração;
- SDLC.

Ferramentas de teste estático

Apoiam:

- revisões;
- análise estática.

Ferramentas de projeto/implementação

Ajudam a produzir:

- casos;
- dados;
- procedimentos.

Ferramentas de execução/cobertura

Executam testes e medem cobertura.

Ferramentas não funcionais

Ajudam a realizar testes difíceis de executar manualmente.

Ferramentas DevOps

Apoiam:

- pipeline;
- CI/CD;
- build;
- fluxo de trabalho.

Ferramentas de colaboração

Apoiam comunicação.

Escalabilidade/padronização

Ex.:

- máquinas virtuais;
- containers.

A questão 39 do Simulado B faz exatamente uma associação entre descrições e categorias de ferramentas.

8.2 Automação de testes

Benefícios:

- economizar tempo;
- reduzir trabalho repetitivo;
- aumentar consistência;
- aumentar repetibilidade;
- obter medidas;
- acelerar feedback;
- reduzir tempo de execução;
- permitir que testadores dediquem mais tempo a testes mais profundos.

Mas:

automação não elimina a necessidade do pensamento crítico humano.

Riscos:

- expectativas irreais;
- custos subestimados;
- manutenção;
- uso inadequado;
- dependência de ferramenta;
- dependência de fornecedor;
- incompatibilidade;
- abandono de ferramenta;
- excesso de confiança na automação.

A questão 40 do Simulado B trabalha justamente a identificação de um benefício da automação.

9. O QUE OS DOIS SIMULADOS ESTÃO TENTANDO ENSINAR

Essa é provavelmente a parte mais importante da sua revisão.

Os simulados não são simplesmente duas provas aleatórias.

Existe uma estrutura muito clara.

Parte 1 — Fundamentos

Os conjuntos variam os objetivos específicos.

Por exemplo:

Set A

- objetivo de teste;
- por que testar;
- princípios;
- atividades;
- contexto;
- papéis;
- habilidades;
- equipe.

Set B

- por que testar;
- QA/QC;
- princípios;
- atividades;
- contexto;
- rastreabilidade;
- habilidades;
- equipe.

Então:

não estude apenas as questões específicas; estude todo o Capítulo 1.

Parte 2 — SDLC

A mesma coisa.

O Set A enfatiza:

- boas práticas;
- TDD/ATDD/BDD;
- shift-left;
- retrospectivas;
- níveis;
- regressão.

O Set B muda alguns pontos:

- impacto do SDLC;
- boas práticas;
- TDD/ATDD/BDD;
- DevOps;
- níveis;
- regressão.

Portanto, memorizar:

“A questão 10 é ATDD”

é muito menos útil que saber:

“Eu sei explicar TDD, BDD e ATDD e diferenciar os três.”

10. A PARTE MAIS IMPORTANTE: QUESTÕES K3

Os simulados deixam muito claro que algumas matérias devem ser **praticadas**, não somente lidas.

As quatro técnicas caixa-preta aparecem fortemente:

Tema	Set A	Set B
EP	Q20	Q20
BVA	Q21	Q21
Tabela de decisão	Q22	Q22
Transição de estado	Q23	Q23

Todas K3.

Depois:

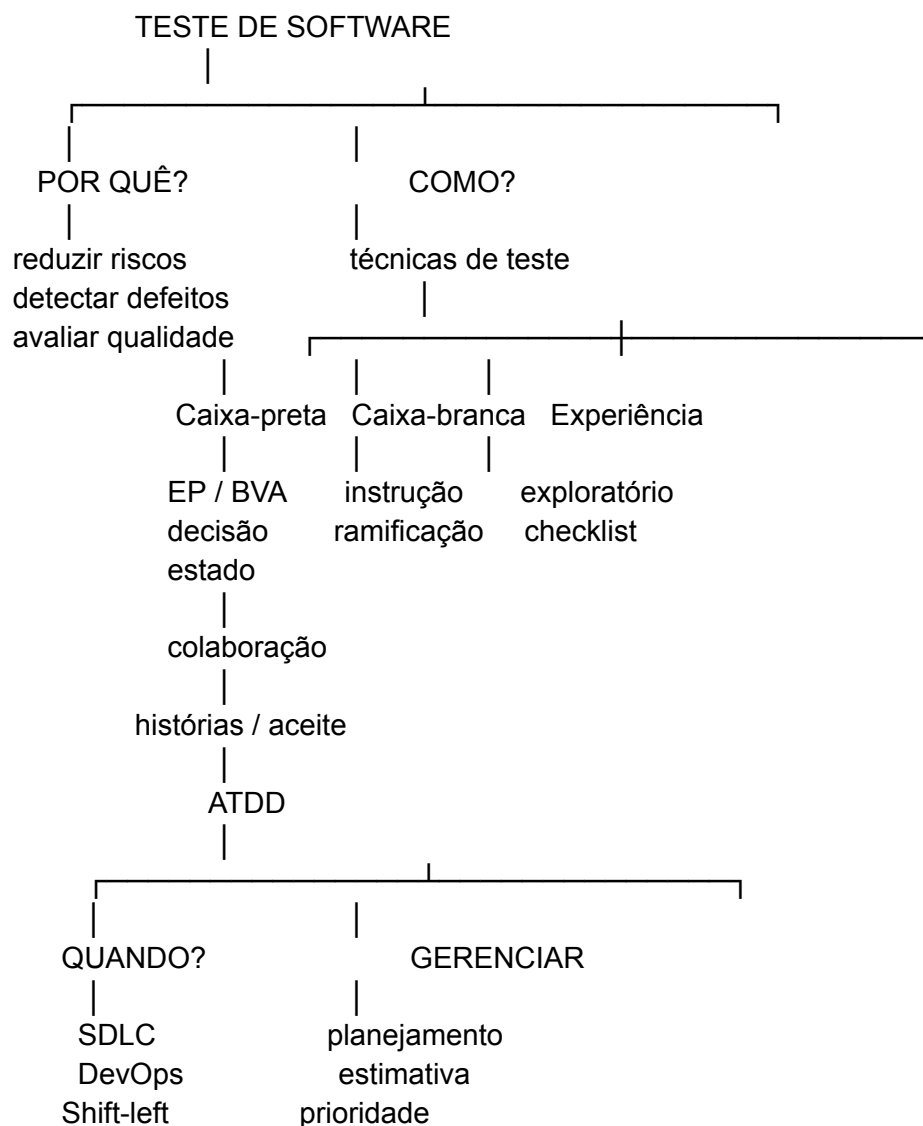
Tema	Set A	Set B
Estimativa	Q32	Q31
Priorização	Q33	Q32
Quadrantes	Q34	Q33
Risco	Q35	Q34
Métricas	Q36 — comunicação	Q35

Comunicação	Q36	Q36
o		
Configuração	Q37	Q37
Defeitos	Q38	Q38

Ou seja, você precisa conseguir **resolver situações**, não simplesmente repetir definições.

11. O MAPA MENTAL GERAL DA MATÉRIA

Tente guardar esta estrutura:



retrospectiva	risco
	métricas
níveis de teste	CM
tipos de teste	defeitos
manutenção	

12. O QUE VOCÊ PRECISA SABER DIFERENCIAR SEM HESITAR

Essas duplas/trincas merecem atenção especial porque aparecem repetidamente em questões.

Teste × depuração

Teste encontra/avalia.

Depuração encontra causa e corrige.

QA × QC

QA = processo/prevenção.

QC = produto/correção.

Teste = parte do QC.

Erro × defeito × falha

Erro humano

↓

Defeito

↓

Falha

Verificação × validação

Verificação = requisitos especificados.

Validação = necessidades dos stakeholders.

Análise × modelagem

Análise = o que testar.

Modelagem = como testar.

Confirmação × regressão

Confirmação = corrigiu?

Regressão = quebrou outra coisa?

Funcional × não funcional

Funcional = o que faz.

Não funcional = quão bem faz.

Caixa-preta × caixa-branca

Preta = comportamento/especificação.

Branca = estrutura interna.

EP × BVA

EP = partições.

BVA = limites.

Instrução × ramificação

Instrução = executar instruções.

Ramificação = executar caminhos/ramificações de controle.

Risco de projeto × produto

Projeto = risco para entregar o projeto.

Produto = risco relacionado à qualidade do produto.

Severidade × prioridade

Severidade = impacto.

Prioridade = urgência de correção.

DoR × DoD

DoR = pronto para começar.

DoD = pronto para terminar.

13. UMA LINHA DE ESTUDO COESA PARA VOCÊ

Eu estudaria esse conteúdo exatamente nesta sequência:

Etapa 1 — Fundamentos

Leia **páginas 14–20**.

Objetivo:

entender o vocabulário da matéria.

Não avance sem dominar:

teste
depuração
QA
QC
erro
defeito
falha
causa-raiz
7 princípios
atividades
testware
rastreadibilidade
papéis
habilidades

Etapa 2 — Ciclo de vida

Leia **páginas 23–28**.

Objetivo:

entender onde os testes entram no desenvolvimento.

Domine:

SDLC
TDD
ATDD
BDD
DevOps
CI/CD

Shift-left
retrospectivas
níveis
tipos
confirmação
regressão
manutenção

Etapa 3 — Teste estático

Leia **páginas 30–33**.

Objetivo:

saber diferenciar teste estático, dinâmico e revisão.

Depois domine:

produtos revisáveis
benefícios
feedback
processo de revisão
papéis
walkthrough
revisão técnica
inspeção
fatores de sucesso

Etapa 4 — Técnicas

Leia **páginas 35–41** com mais cuidado.

Aqui você deve realmente praticar:

EP
BVA
Tabela de decisão
Transição de estado
Instrução
Ramificação
Suposição de erro
Exploratório
Checklist
História de usuário
INVEST
Critérios de aceite
ATDD

Especialmente:

EP, BVA, tabela de decisão e transição de estado.

Os simulados colocam esses quatro como questões K3.

Etapa 5 — Gerenciamento

Leia **páginas 42–51**.

Aqui você deve saber aplicar:

Plano de teste
DoR / DoD
Estimativa
3 pontos
Priorização
Dependências
Pirâmide
Quadrantes
Risco
Métricas
Relatórios
Comunicação
CM
Relatório de defeito

Etapa 6 — Ferramentas

Leia **páginas 53–54**.

Concentre-se em:

categorias de ferramentas
benefícios da automação
riscos da automação

14. O QUE NÃO FAZER AO ESTUDAR

Pelos próprios simulados, eu evitaria três abordagens.

Não memorize as letras

Exemplo:

“Questão 20 = alternativa A.”

Isso não ajuda.

O Set A pode ter:

Q20 → B

enquanto o Set B pode mudar.

O que permanece é:

Q20 → EP → aplicar a técnica

Os gabaritos comprovam que os conteúdos são reorganizados entre os conjuntos, mesmo mantendo a mesma estrutura geral de prova.

Não memorize frases isoladas

Por exemplo:

“Shift-left é testar cedo.”

É correto, mas insuficiente.

Você precisa reconhecer **qual atividade representa shift-left**.

Não estude apenas definição

Especialmente no Capítulo 4 e Capítulo 5.

Você precisa saber:

ler um cenário → identificar o problema → selecionar o conceito → aplicar a técnica.

15. CHECKLIST DE DOMÍNIO

Em vez de fazer outro simulado agora, use esta lista como critério de estudo.

Você terminou o Capítulo 1 quando consegue explicar sem consultar:

O que é teste?

Teste e depuração são iguais?

O que QA faz?

O que QC faz?
Qual a sequência erro → defeito → falha?
Quais são os 7 princípios?
O que acontece na análise de teste?
E na modelagem?
Qual a diferença entre testador e gerenciamento de testes?

No Capítulo 2:

O que muda nos testes dependendo do SDLC?
TDD, BDD e ATDD são iguais?
O que DevOps muda?
O que significa shift-left?
O que uma retrospectiva produz?
Quais são os cinco níveis de teste?
Funcional e não funcional?
Confirmação e regressão?
Quando ocorre teste de manutenção?

No Capítulo 3:

O que pode ser testado estaticamente?
Estático e dinâmico?
Quais são os benefícios?
Quais etapas existem numa revisão?
Quem são os papéis?
Informal, walkthrough, técnica e inspeção?

No Capítulo 4:

Qual técnica usar para partições?
Qual para limites?
Qual para combinações de condições?
Qual para estados?
Qual mede instruções?
Qual mede ramificações?
Quando usar teste exploratório?
O que é checklist?
O que significa INVEST?
Qual formato de critério é orientado a cenários?
Como ATDD funciona?

No Capítulo 5:

Entrada e saída?
DoR e DoD?
Como calcular estimativa por proporção?
Como usar três pontos?
Como priorizar testes?

O que fazer quando existem dependências?
O que é a pirâmide?
O que são os quatro quadrantes?
Risco de projeto e produto?
Probabilidade × impacto?
Métricas?
Relatórios?
CM?
Como escrever um relatório de defeito?

No Capítulo 6:

Quais tipos de ferramentas existem?
Quais benefícios a automação oferece?
Quais riscos ela traz?

16. RESUMO FINAL PARA GUARDAR NA CABEÇA

A matéria inteira pode ser reduzida a esta sequência lógica:

Testamos porque software pode apresentar problemas e porque precisamos reduzir riscos e avaliar sua qualidade.

Os testes precisam acontecer ao longo do desenvolvimento, e não somente no final.

Podemos testar estaticamente, sem executar o software, ou dinamicamente, executando-o.

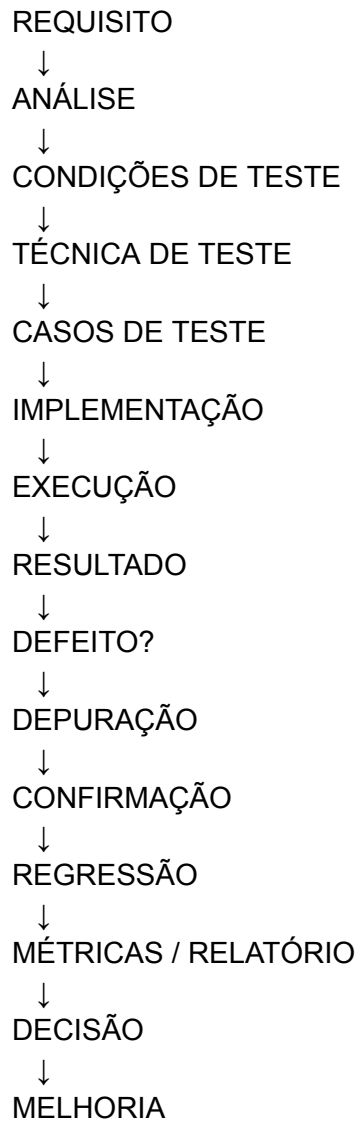
Para derivar bons testes usamos técnicas: caixa-preta, caixa-branca e experiência.

EP divide em grupos; BVA olha limites; tabela de decisão trabalha combinações; transição de estado trabalha estados/eventos.

Depois precisamos administrar o trabalho: planejar, estimar, priorizar, controlar riscos, medir, comunicar, controlar configurações e gerenciar defeitos.

Finalmente, ferramentas ajudam a executar e administrar tudo isso, mas automação não substitui o pensamento crítico.

E existe uma sequência que conecta praticamente toda a matéria:



Essa é a linha que conecta os seis capítulos.

O mais importante para sua preparação é não tratar os simulados como 80 questões isoladas. Eles estão mostrando, na prática, quais conceitos você precisa reconhecer e quais precisa saber aplicar. Os gabaritos ainda fornecem o **Objetivo de Aprendizagem (LO)** e o **nível cognitivo**, então eles funcionam praticamente como um mapa do tipo de conhecimento esperado.